Singly Linked List –
Reverse List
and Time Complexity.

Program: –

```
void reverseList(Node **head)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }
    Node *previous = nullptr;
    Node *current = *head;
    Node *next = nullptr;

    while (current != nullptr)
    {
        next = current->next;
        current->next = previous;
        previous = current;
        current = next;
    }

    *head = previous;
}

…….

case 14:
        {
            reverseList(&head);
            cout << "Reverse List:" << endl;
            break;
        }
```

Program Analysis:

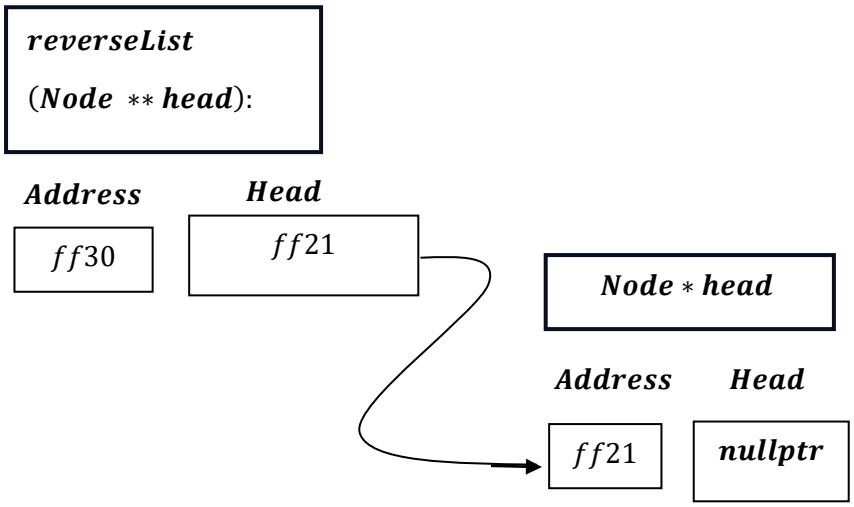
Now, as we call the function :

```
        reverseList(&head);
```

Say, reverseList(&***head***: ***ff21***);

But our list is Empty, hence :

```
void reverseList(Node **head) {…}
```



Now if Head pointer of Node structure points to nullptr then list is empty ,

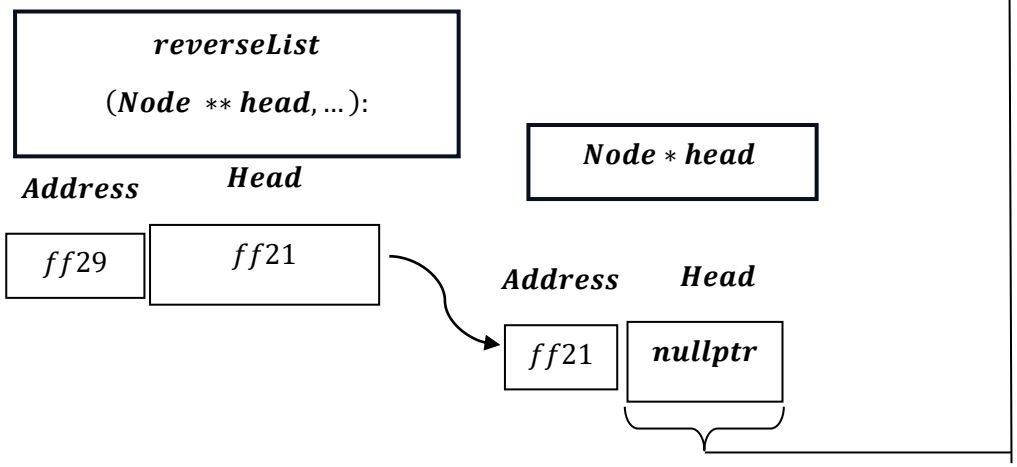
i.e. Head carries value nullptr (***not pointing to any valid memory location***).

```
    if (*head == nullptr)
    {
```

```
        cout << "Error: List is empty." << endl;
        return;
    }
```

$if(*head == nullptr)$ *if it is true*

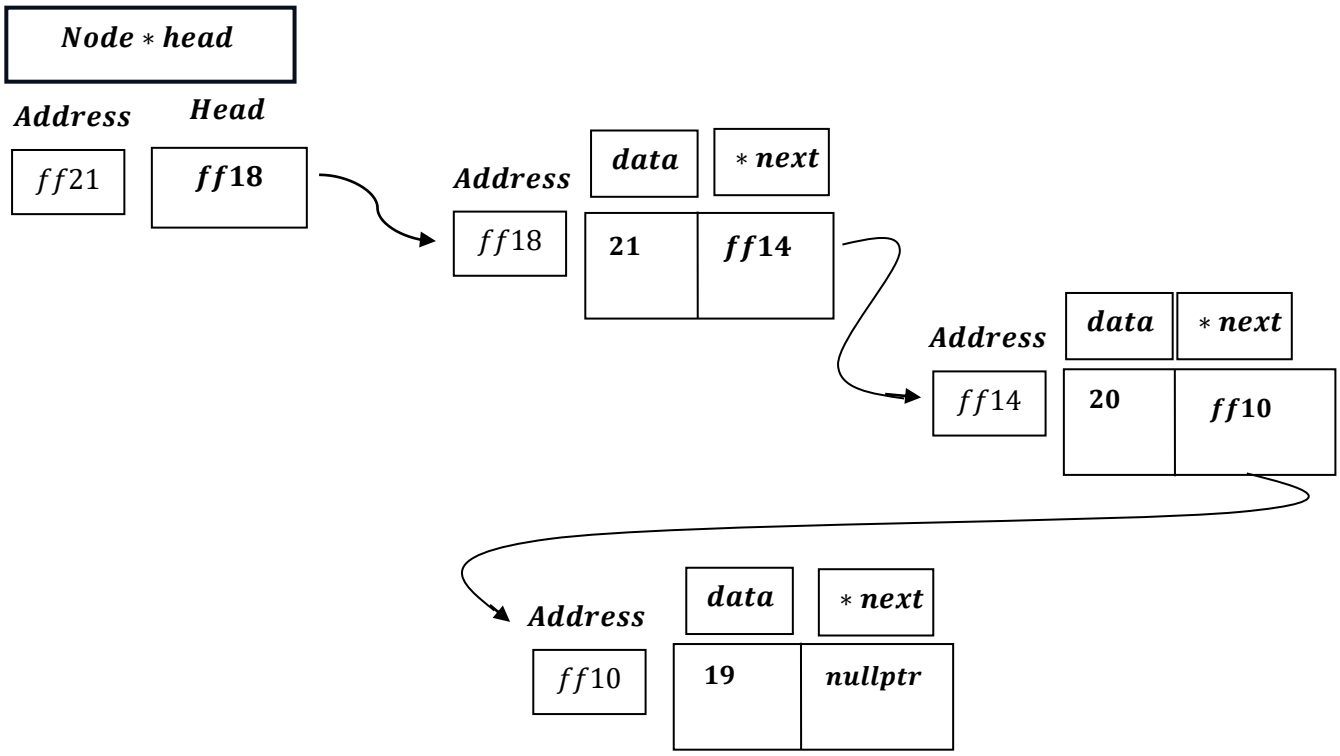
If true then Why true ?
*Ans: Though * head which is of Node ** head*
temporary pointer to pointer variable of the function
*, as * head is used , it dereference from Node ** head and*
*refer to Node * head global pointer to which Node ** head*
is pointing to and then its value is checked is
whether equal to nullptr or not ? This occurs during execution
and compilation of the reverseList function.



Then , it prints a message : `Error: List is empty.`

Now, List is not Empty

We are given with a Linked List:



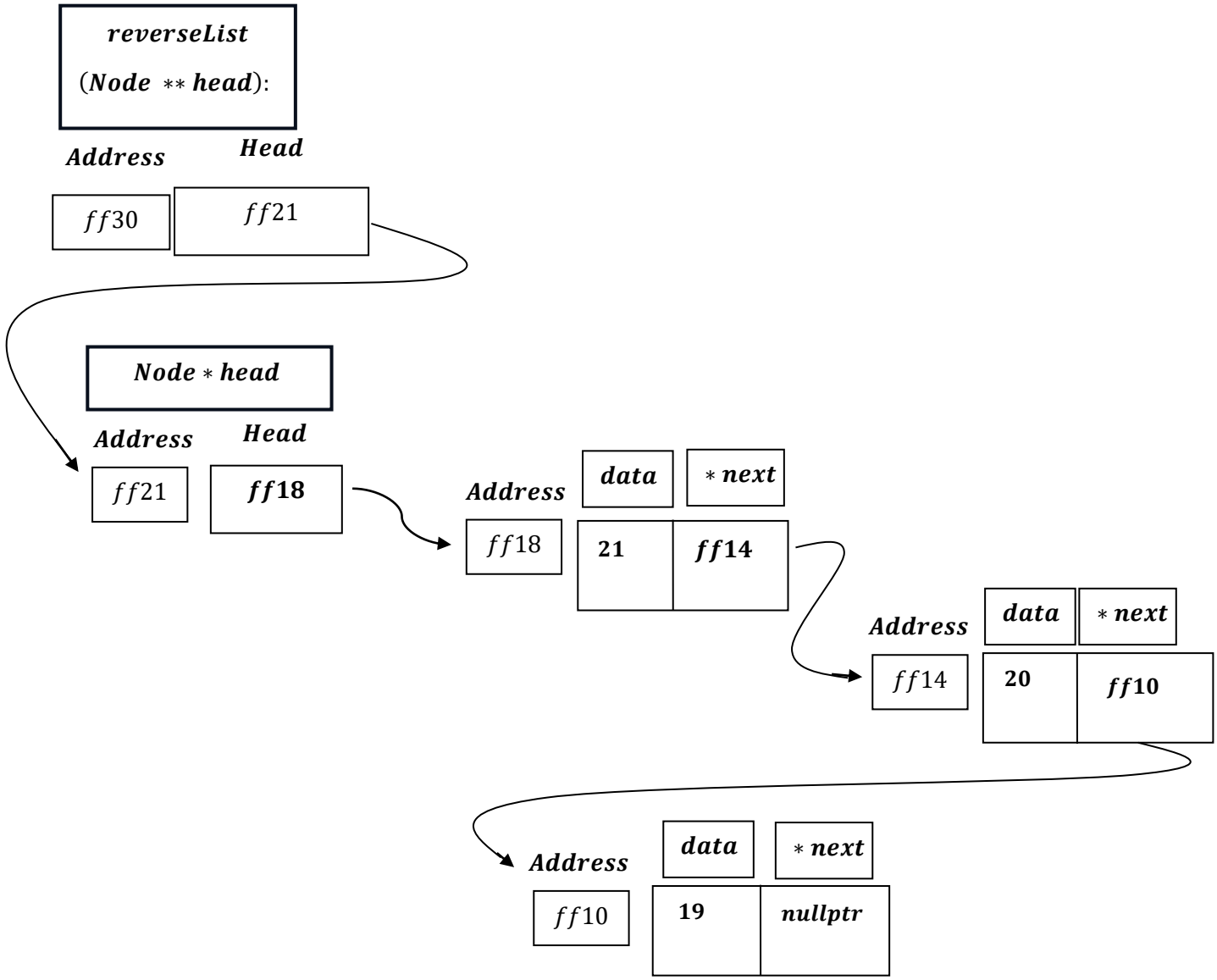
Next Function Call: ***reverseList***(&***head***)

```
reverseList(&head);
```

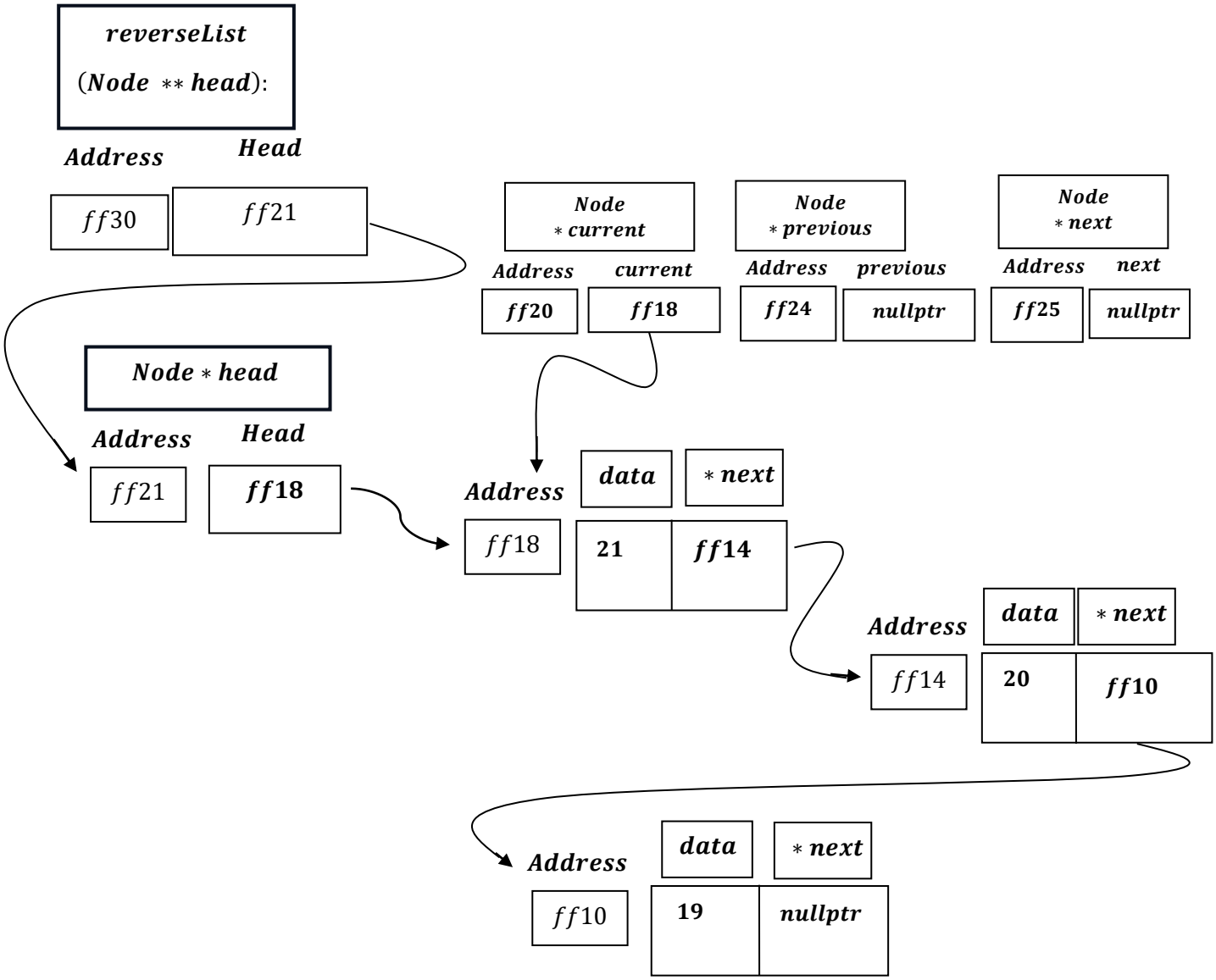
i.e.reverseList(&***head***: ***ff21***);

Now:

```
void reverseList(Node **head) {…}
```




```
Node *previous = nullptr;
Node *current = *head;
Node *next = nullptr;
```



And now:

```
while (current != nullptr)
    {
        next = current->next;
        current->next = previous;
        previous = current;
        current = next;
    }
```

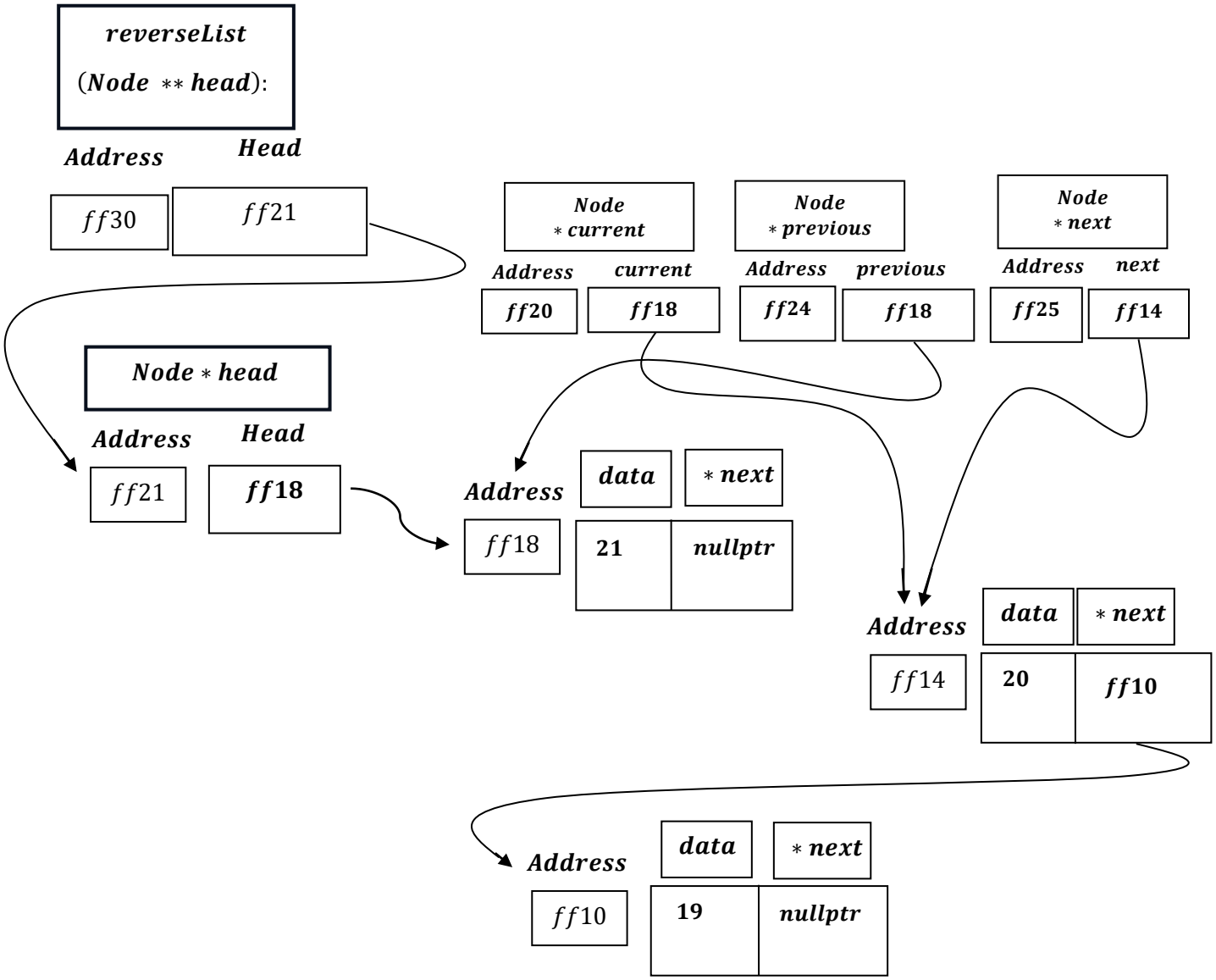
As current is not `nullptr` hence:

$next = current \rightarrow next = ff14;$

$current \rightarrow next = previous = nullptr;$

$previous = current = ff18$

$current = next = ff14$

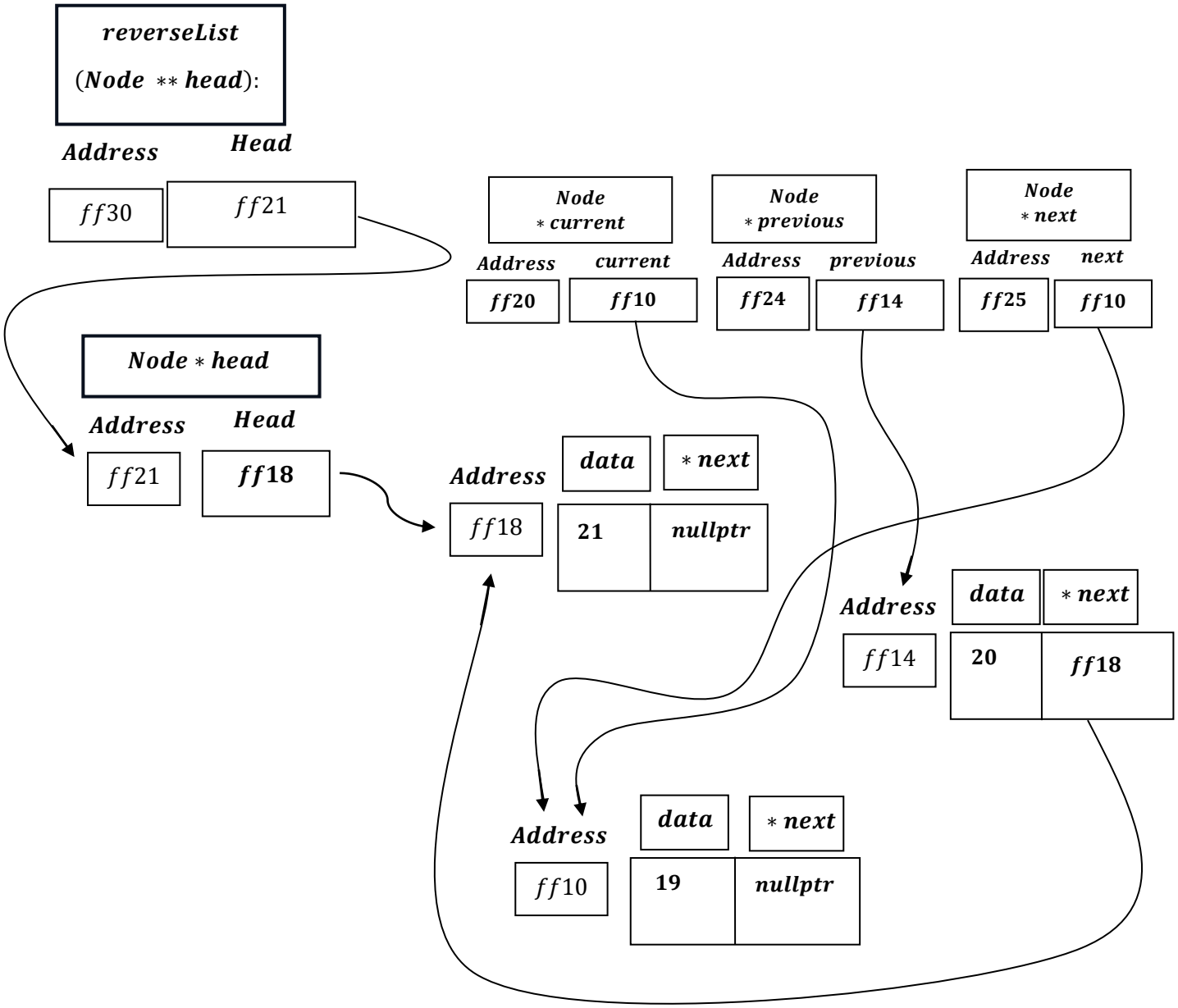


$next = current \rightarrow next = ff10;$

$current \rightarrow next = previous = ff18\ ;$

$previous = current = ff14$

$current = next = ff10$

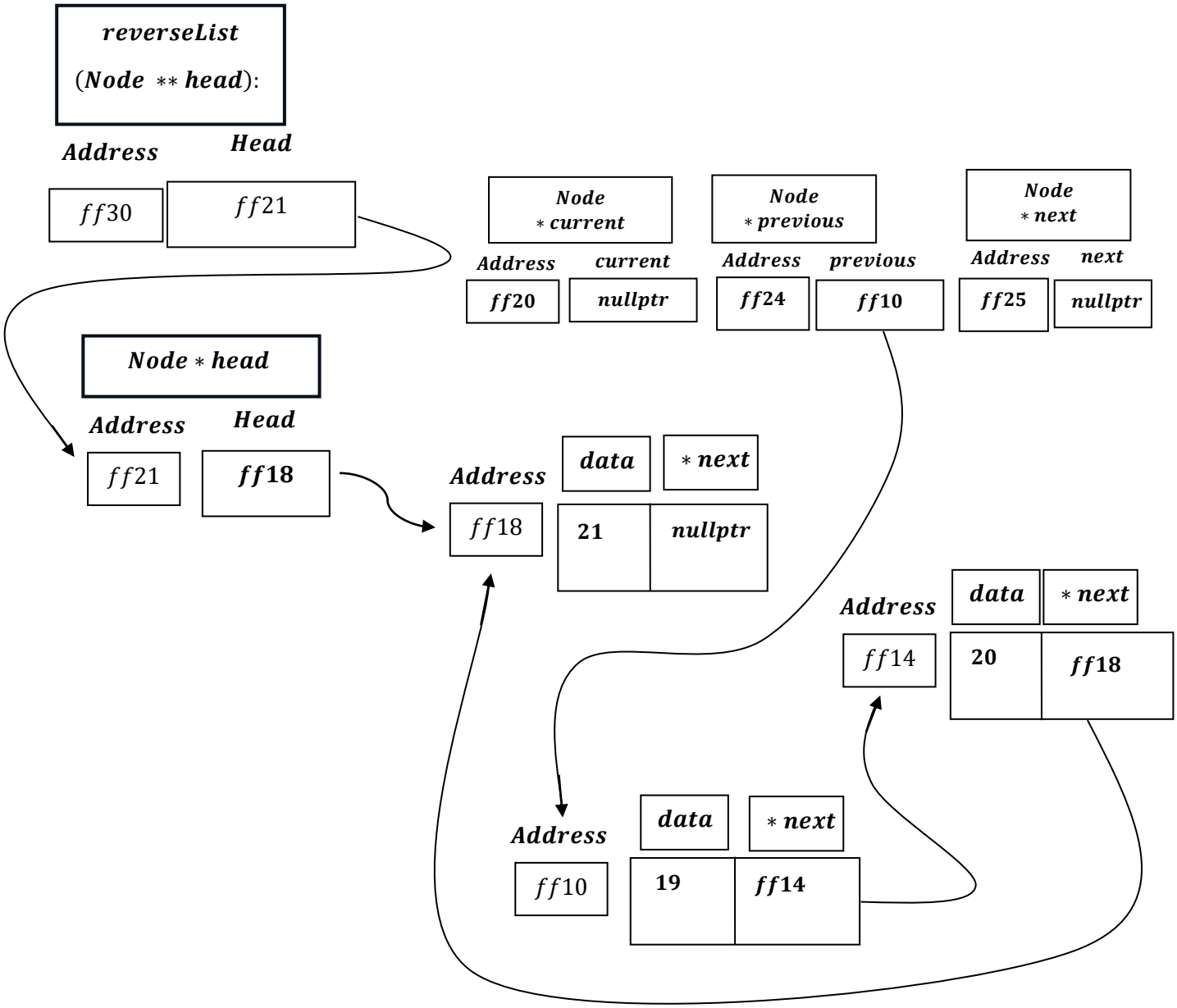


$next = current \rightarrow next = nullptr;$

$current \rightarrow next = previous = ff14$;

$previous = current = ff10;$

$current = next = nullptr;$

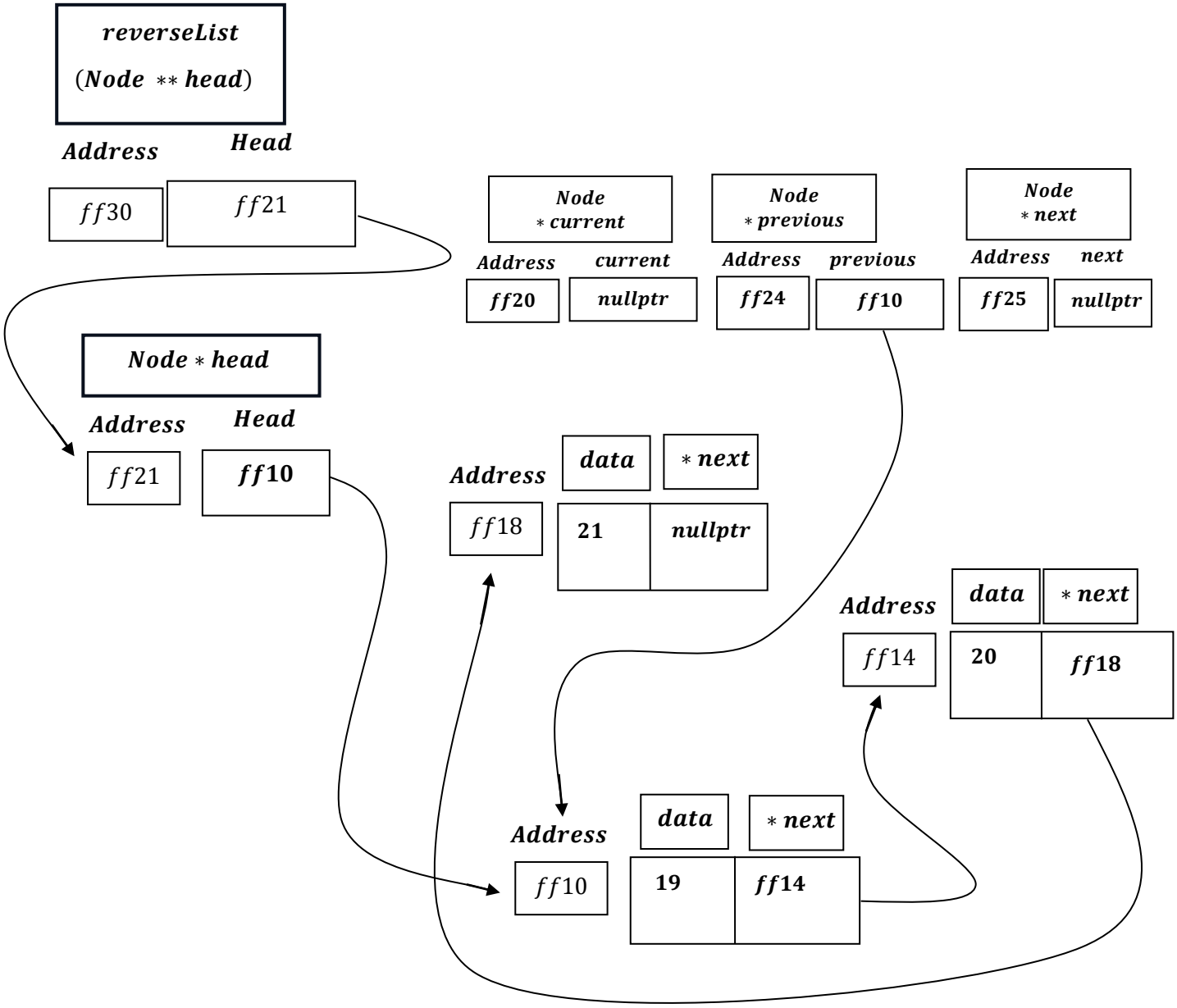


Now , as current = nullptr ; the loop terminates , and after loop gets terminated:

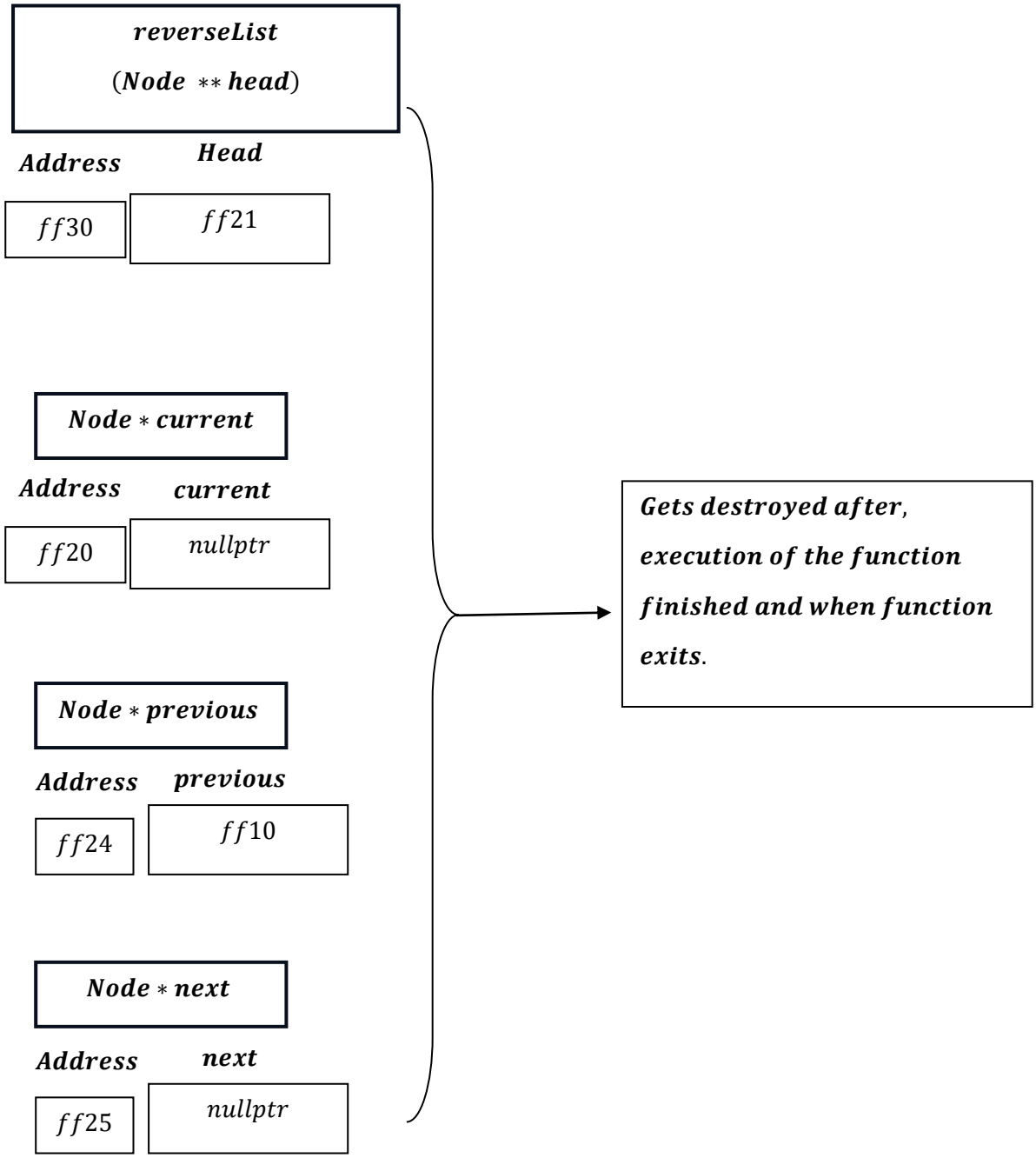
```
while (current != nullptr){….}

*head = previous;
```

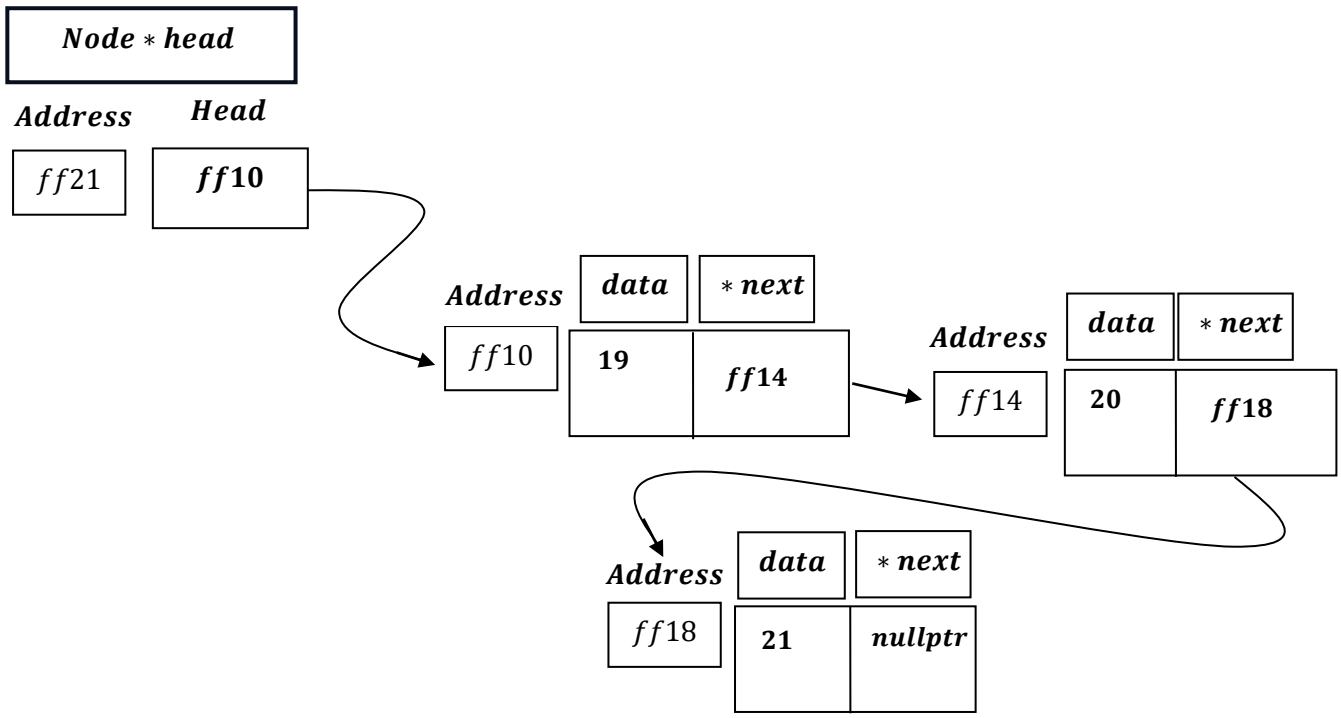
$*\ head = previous = ff10;$



Now as the function ends , all the temporary pointer variables gets destroyed:



And we are left with:



Reverse List Function Step – by – Step Explanation:

→ ***If the list is empty (`* head` is `nullptr`), print an error message and return from the function.***

–***previous***: ***Initially set to `nullptr`. This will eventually become the new head of the reversed list.***

– ***current***: ***Initially set to the current head of the list (`* head`). This pointer will traverse the list.***

– ***next***: ***Initially set to `nullptr`. This will temporarily store the next node during the reversal process.***

– ***Loop Condition***: ***The loop continues until `current` becomes `nullptr`, indicating the end of the list.***

– ***Inside the Loop***:

- `next = current-> next;`: Store the next node in the list.

- `current → next = previous;`: Reverse the current node's `next` pointer to point to the previous node.

- `previous = current;`: Move the `previous` pointer to the current node.

- `current = next;`: Move the `current` pointer to the next node (stored in `next`).

- Update the original head pointer (`*head`) to point to `previous`.

Time Complexity of Reverse List

```
void reverseList(Node **head)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }
    Node *previous = nullptr;
    Node *current = *head;
    Node *next = nullptr;

    while (current != nullptr)
    {
        next = current->next;
        current->next = previous;
        previous = current;
        current = next;
    }

    *head = previous;
}
```

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

$\rightarrow O(1)$

runs `n` times

Inner Statements in Loop also run `n` times.

$\rightarrow O(1)$

Best Case(When List is Empty)

$\rightarrow$ *If the list is empty (`* head` is `nullptr`), print an error message and return from the function.*

This takes $O(1)$ *time or* $\Omega(1)$ *time* .

$\underline{Worst\ Case(When\ List\ is\ not\ Empty)}$

$\rightarrow$ *If the list is empty (`* head` is `nullptr`), print an error message and return from the function. Though List is not empty here hence Checking takes $O(1)$ constant time.*

−previous: Initially set to `nullptr`. This will eventually become the new head of the reversed list. This operation takes $O(1)$ constant amount of time.

− current: Initially set to the current head of the list (`* head`). This pointer will traverse the list. This operation takes $O(1)$ constant amount of time.

− next: Initially set to `nullptr`. This will temporarily store the next node during the reversal process . This operation takes $O(1)$ constant amount of time.

− Loop Condition: The loop continues until `current` becomes `nullptr`, indicating the end of the list.

− Inside the Loop:

- *− `next = current−> next;`: Store the next node in the list.*
- *− `current → next = previous;`: Reverse the current node's `next` pointer to point to the previous node.*
- *−`previous = current;`: Move the `previous` pointer to the current node.*
- *− `current = next;`: Move the `current` pointer to the next node (stored in `next`).*

Note: Each of the inner statement run $1 + 1 + 1 + .. n$ times , hence 3 inner loop statement will run $3n$ times or $O(3n)$ is $O(n)$, removing the constant.

$-$ *Update the original head pointer* (`$*$ head`)*to point to* `previous`*. This operation takes* $O(1)$ *constant amount of time.*

Hence, total complexity: $O(1) + O(1) + O(1) + O(1) + O(n) + O(1) = O(n)$.

Hence , Worst Case time complexity is : $O(n)$.

Average Case

We can say , if List is Empty , we get our best case scenario, and when list is not empty , we get our worst case scenario. But though best case and worst case scenarios are present as, always we can determine list get traversed from beginning to end , hence we can say average case is our worst case here i.e. always the entire list is traversed i.e. $O(n) = \Theta(n)$.

This means the execution time is solely dependent on the length of the list. If the list is empty, the execution time is constant (best case). If the list is not empty, the execution time is proportional to the length of the list (worst case). Since the loop rotation is predetermined, there's no element of randomness or varying input data to consider. Therefore, the concept of an average case, where we average the execution time across different input possibilities, doesn't strictly apply here. Hence Reverse List doesn't have a true average case.

$Note\ , by\ relation: \Omega \leq \Theta \leq O\ , we\ get:$

$\Longrightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$

$\Longrightarrow c_1 \times 1 \leq N \leq c_2 \times N$

$Singly\ Linked\ List$	$Cases$	$Time\ Complexity$
$Reverse\ List$	$1. Best\ Case$	$\Omega(1)$
	$2. Worst\ Case$	$O(n)$
	$3. Average\ Case\ (= Worst\ Case)$	$\Theta(n)$